

CST8246 - Network Tools and Automated Firewall Configuration

Objectives

- Utilize `ss` to list service statuses.
- Employ `netcat` to establish a basic client/server setup.
- Use `iptables` to filter network traffic and manage service access.
- Apply `tcpdump` to capture and analyze network traffic.
- Automate firewall configuration by creating a script to manage `iptables` rules dynamically.

Lab Outcomes

- Gain proficiency in network tools that support service management.
- Develop a script to automate firewall rule implementation, reinforcing practical scripting skills.

Lab Deliverables

- Implement controlled access to a specified service port.
- Create and execute a script to configure `iptables` rules according to lab requirements.
- Detailed demonstration requirements are available on Brightspace where this lab was downloaded.

Section A: Working with `netstat/ss`

Overview

- The utilities `netstat` and `ss` are used to verify service connections. We will use these utilities to check which services are currently running on our systems and on which interfaces and ports they are listening.

Note: While the `service` command can verify if a service is running, it does not provide sufficient information to confirm whether a service is available as intended (i.e., as a network service application).

- The options that are useful for this course are `-tpan`.
- The `ss` utility serves as a replacement for `netstat`; both use mostly the same options.

Examples

Example #1: Service in Listening State

To verify if a service is in a listening state, indicating it's ready to accept incoming connections, you can use the following commands:

Using `ss`:

```
[root@localhost ~]# ss -tpan | grep ":22"

LISTEN 0 128 *:22 *:~ users:(("sshd",1245,3))
```

Using `netstat`:

```
[root@localhost ~]# netstat -tpan | grep ":22"

tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 1245/sshd
```

In both outputs, the service (`sshd`) is listening on port 22 across all available interfaces (0.0.0.0). The process ID (PID) is 1245. This confirms that the SSH service is ready to accept incoming connections.

Example #2: Service in Established State

To check if a service has an established connection with a client, you can use:

Using `ss`:

```
[root@localhost ~]# ss -tpan | grep ":22"

ESTAB 0 0 172.16.30.101:22 172.16.31.101:49194 users:(("sshd",5018,3))
```

Using `netstat`:

```
[root@localhost ~]# netstat -tpan | grep ":22"

tcp 0 0 172.16.30.101:22 172.16.31.101:49194 ESTABLISHED 5018/sshd
```

These outputs indicate that the SSH service (sshd) on the server (172.16.30.101:22) has an established connection with a client at 172.16.31.101 on port 49194. The PID of the sshd process handling this connection is 5018.

Note: For concise outputs, especially when dealing with short command results, using pipes with `grep` might be unnecessary. Both `ss` and `netstat` provide options to filter results directly. For instance, to list only listening SSH connections, you can use:

Using `ss`:

```
[root@localhost ~]# ss -tlnp | grep ":22"
```

Using `netstat`:

```
[root@localhost ~]# netstat -tlnp | grep ":22"
```

These commands list all listening (-l) TCP (-t) sockets without resolving names (-n), and display the process using the socket (-p).

Section B: Working with Netcat (nc)

Netcat as a Client: Protocol Communication (Telnet Style)

Overview

Netcat (nc) is a versatile networking utility that reads and writes data across network connections using the TCP/IP protocol. When provided with the necessary arguments—specifically, the hostname (or IP address) and the port number—Netcat establishes a TCP connection to the specified service. This allows you to interact directly with the service by typing commands using the appropriate protocol language.

Example

To connect to an HTTP server on port 80 and send an HTTP request:

```
[root@localhost ~]# nc www.example101.lab 80
```

After establishing the connection, you can manually input HTTP commands. For example, to retrieve the homepage, you might type:

```
GET / HTTP/1.1
```

```
Host: www.example101.lab
```

After pressing Enter twice, the server should respond with the HTML content of the homepage.

Note: Netcat can only maintain one connection at a time.

Additional Uses of Netcat

Beyond acting as a client for protocol communication, Netcat offers a range of functionalities:

- **Port Scanning:** Netcat can scan for open ports on a target machine. For example, to scan ports 20 through 25 on a host:

```
nc -v -w 2 -z 192.168.56.1 20-25
```

Here, -v enables verbose mode, -w 2 sets a timeout of 2 seconds, and -z sets Netcat to scanning mode without sending data.

- **File Transfers:** Netcat can transfer files between hosts. To send a file from a client to a server:

On the receiving (server) side:

```
nc -l -p 1234 > received_file
```

On the sending (client) side:

```
nc 192.168.1.9 1234 < file_to_send
```

Replace 192.168.1.9 with the server's IP address, 1234 with the chosen port number, file_to_send with the name of the file you're sending, and received_file with the desired name for the received file.

- **Chat Application:** Netcat can facilitate a simple chat between two computers. On one machine, set up Netcat to listen on a port:

```
nc -l -p 1234
```

On the other machine, connect to the first machine's IP address on the same port:

```
nc 192.168.1.9 1234
```

Now, anything typed in one terminal will appear in the other, enabling a basic chat functionality.

Netcat's flexibility makes it a valuable tool for network administrators and developers alike, offering capabilities that range from simple data transfers to more complex network debugging and exploration tasks.

Netcat as a Server

Overview

Previously, we explored using Netcat in client mode. However, Netcat can also operate in server mode, listening for incoming connections. By establishing a client-server connection with Netcat, you can verify the ability to establish a TCP connection between two systems.

Syntax

- **On the server:**

```
nc -vl [host] 1234
```

This command initiates Netcat in listening mode (-l) on port 1234, with verbose output (-v). The [host] parameter specifies the network interface to bind to; if omitted, Netcat listens on all available interfaces.

- **On the client:**

```
nc -v host 1234
```

This command directs Netcat to connect to the specified host (IP address or hostname) on port 1234, with verbose output (-v).

Exercise 1: Chatting Using a Netcat Client-Server Connection

1. **Set up a server-client connection on an unused port and verify data transfer:**

- **Select a port number:**

- **Well-known ports:** These are listed in /etc/services.
- **Currently used (bound) ports:** Use the ss command to display these.

- **Choose an interface:**

You can use your assigned IP address, the loopback address (127.0.0.1), your hostname, or the localhost hostname. Ensure both the server and client use the same interface argument.

- **Start the Netcat server:**

```
nc -vl 1234
```

This command starts Netcat in listening mode on port 1234 with verbose output.

- **Connect the Netcat client:**

```
nc -v [server_host] 1234
```

Replace [server_host] with the server's IP address or hostname.

- **Verify the connection state with ss:**

On both the server and client machines, run:

```
ss -tpan | grep 1234
```

This command checks the status of the TCP connection on port 1234.

- **Test data transfer:**

Type messages on either the server or client terminal to ensure they are received on the opposite end.

- **Terminate the connection:**

Press [Ctrl-C] on either terminal to close the connection.

Exercise 2: Using Different Interface Arguments for Server and Client

1. Set up the server on one interface:

- For example, to use localhost:

```
nc -vl localhost 1234
```

- **Verify the server is listening on the selected interface:**

```
ss -tpan | grep 1234
```

2. Attempt to connect the client using a different server interface:

- For example, using the server's IP address on the red network:

```
nc -v 172.16.30.MN 1234
```

Replace MN with your assigned network number.

- **Expected outcome:**

You should receive a "connection refused" error message. This occurs because the server is bound to localhost, making it inaccessible via other interfaces.

These exercises demonstrate Netcat's versatility in establishing TCP connections and highlight the importance of interface selection when setting up server-client communications.

Netcat (nc) can be utilized to transfer files between a "server" and a "client" system by establishing a direct connection. Here's how you can set it up:

On the System Providing the File (the "File Server"):

Use the following command to listen on a specific port and send the desired file:

```
nc -l -p 1234 < file1
```

This command does the following:

- `-l`: Puts Netcat in listening mode, waiting for incoming connections.
- `-p 1234`: Specifies the port number (1234 in this example) on which Netcat listens.
- `< file1`: Redirects the content of file1 to be sent over the established connection.

On the System Receiving the File (the "Client"):

Use the following command to connect to the server and receive the file:

```
nc [server_host] 1234 > file2
```

This command does the following:

- `[server_host]`: Replace this with the IP address or hostname of the server providing the file.
- `1234`: Specifies the port number to connect to, which should match the server's listening port.
- `> file2`: Redirects the received data into file2 on the client system.

Important Considerations:

- **Port Selection:** Ensure that the chosen port (e.g., 1234) is not in use by other services and is open on any firewalls between the client and server.
- **File Permissions:** Verify that the user executing the Netcat command has the necessary permissions to read file1 on the server and write to file2 on the client.
- **Network Security:** Be cautious when using Netcat over untrusted networks, as the data is transferred in plain text without encryption. For secure transfers, consider using tools like scp or setting up an encrypted tunnel.

By following these steps, you can effectively transfer files between systems using Netcat.

Section C: Working with iptables

Exercise Overview

This section provides a procedural overview; the specific exercises are detailed in the "iptables Scenarios" section.

Default Policy

By default, iptables is configured to accept all traffic. In this lab, we will maintain this default policy and introduce specific rules to reject selected traffic.

Server Setup

- **Initiate the Server:** Start the server using nc (netcat) in "server" mode.
- **Verify Listening Status:** Ensure the server is listening on all interfaces and the designated port. You can use utilities like ss or netstat to verify this.
- **Port Selection:** For this lab, we will use port 49999. To ensure this port is neither assigned nor currently in use:
 - **Check Assigned Ports:** Consult the /etc/services file, which lists commonly assigned port numbers.
 - **Verify Port Usage:** Use ss or netstat to confirm that no service is currently bound to port 49999.

Testing

- **Test Plan:** A test plan is provided for each exercise.
- **Client Connections:** Utilize nc in "client" mode to connect to your server using the parameters specified in the test plan.

iptables Setup

- **Basic Commands:**
 - **Flush Current Rules:** To clear existing rules, use:

```
iptables -F
```
 - **List Current Rules:** To display current rules in the INPUT chain, use:

```
iptables -L INPUT
```

For a more detailed output with numeric values for hosts and ports, use:

```
iptables -L -n -v
```

- **Setting Up Rules:**
 - Implement all rules as specified in the lab exercises. Note that these rules are stored in memory only and will not persist after a reboot.
 - After setting up the rules, list them to verify their correctness.

- **Recording Rules:** For your convenience, document the rules you create, as you may need to re-enter them during the exercises.

Rule Example

To reject all incoming traffic from your client connecting to port 49999, you can add the following rule:

```
iptables -A INPUT -s 172.16.31.MN -p tcp --dport 49999 -j REJECT
```

Explanation of the command:

- **-A INPUT:** Appends the rule to the INPUT chain, affecting incoming traffic.
- **-s 172.16.31.MN:** Specifies the source IP address of the client.
- **-p tcp --dport 49999:** Indicates the protocol (tcp) and the destination port (49999).
- **-j REJECT:** Specifies the target action for matching packets, which is to reject them.

By following these steps, you can effectively manage and test iptables rules to control network traffic as required for your lab exercises.

iptables Scenarios

Before Getting Started: iptables Setup

- **Disable the Default Firewall Service:** If you haven't already, stop the default firewall service. To prevent it from starting automatically, you can mask the service using:

```
systemctl mask firewalld
```

To unmask the service later, use the unmask action.

- **Install iptables-services:** This package enables systemd to manage iptables. After installation, you can control the iptables service with:

```
systemctl [action] iptables
```

Scenario #1: Accept All Incoming Traffic for a Given Service on All Server Interfaces

By default, iptables accepts all incoming traffic on all ports. Therefore, no additional rules are required for this scenario. However, we will verify that iptables is accepting all incoming traffic on a selected port.

Server Setup:

Open four terminal windows:

1. **Netcat Server:** To set up the server to listen on port 49999 on all interfaces, you may want to include the verbose (-v) and "keep-open" (-k) options:

```
nc -vkl 49999
```

2. **Netcat Client on the Server:** Use this terminal to simulate client connections from the server itself.
3. **Socket Statistics (ss):** Monitor socket statistics to observe connection states.
4. **Firewall Rules:** Manage and view iptables rules.

Steps:

1. **Start the Netcat Server:**

In the first terminal, run the following command to start the netcat server listening on port 49999:

```
nc -vkl 49999
```

This command sets up the server to listen on all network interfaces.

2. **Verify the Server's Listening Status:**

In the second terminal, use the ss command to confirm that the server is listening on port 49999 across all interfaces:

```
ss -tuln | grep 49999
```

This command displays listening TCP and UDP ports, filtering for port 49999.

iptables Setup:

1. **List Current iptables Rules:**

In the third terminal, display the current iptables rules:

```
iptables -L -n -v
```

This command provides a verbose, numeric listing of the current rules.

2. **Flush All Rules (If Necessary):**

If there are pre-existing rules that might interfere with this scenario, flush them:

```
iptables -F
```

This command clears all rules in the filter table.

Test Plan:

1. Initiate a Client Connection:

In the fourth terminal, simulate a client connecting to the server:

```
nc [server_ip] 49999
```

Replace [server_ip] with the actual IP address of your server.

2. Verify Connection Establishment:

In the third terminal, use the ss command to check that the connection state is "established":

```
ss -tuln | grep 49999
```

This command should show an established connection on port 49999.

By following these steps, you can confirm that iptables is configured to accept all incoming traffic on the specified port.

Test purpose	Test command on client	Expected test result	Actual test result
Client connection <u>from your server</u> using <i>loopback address</i>	<code>nc 127.0.0.1 49999</code>	OK	
Client connection <u>from your server</u> using <i>static IP</i>		OK	
Client connection <u>from your client</u> using <i>static IP</i>		OK	

Note: Table provided for your convenience only.

Scenario #2: Blocking All Incoming Traffic for a Specific Service on All Server Interfaces

In this scenario, we'll configure the server to reject all incoming traffic on a designated port and verify that this traffic is effectively blocked.

Server Setup:

1. Initialize the Server:

- Configure the server to listen on port 49999 across all network interfaces.

2. Verify Listening Status:

- Ensure the server is actively listening on all interfaces for the specified port.

Iptables Configuration:

1. Review Current Rules:

- Display existing iptables rules to understand the current configuration.

2. Flush Existing Rules (If Necessary):

- Clear all current rules to start with a clean slate:

```
iptables -F
```

3. Add Rule to Reject Incoming Traffic:

- Implement a rule to reject all incoming traffic on port 49999:

```
iptables -A INPUT -p tcp --dport 49999 -j REJECT
```

Note: Using the REJECT target sends a response to the client, informing them that the connection is denied. In contrast, the DROP target silently discards the packet, causing the client to wait until it times out.

4. Confirm the New Rule:

- List the iptables rules to verify the new rule has been added:

```
iptables -L -n -v
```

Test Plan:

- Attempt to connect to the server on port 49999 using a client.
- Monitor the connection status using the ss command to ensure that the server remains in the listening state, as incoming packets are rejected before reaching the application.

By following these steps, we effectively block all traffic to the service on port 49999.

Test purpose	Test command	Expected test result	Actual test result
Client connection <u>from your server</u> using <i>loopback address</i> .		NOK	
Client connection <u>from your server</u> using <i>static IP</i> .		NOK	
Client connection <u>from your client</u> using <i>static IP</i> .		NOK	
Start another server on port 59999. Verify with <i>ss</i> . Client connection <u>from your client</u> using <i>static IP</i> to port 59999.		OK	

Note: Table provided for your convenience only.

Scenario #3: Permit All Incoming Traffic from the Server on All Ports and Interfaces

In this scenario, we aim to ensure that traffic originating from the server itself is allowed across all ports and interfaces, irrespective of other firewall rules.

Server Setup:

1. **Initialize the Server:**
 - Configure the server to listen on port 49999 across all network interfaces.
2. **Verify Listening Status:**
 - Ensure the server is actively listening on the specified port on all interfaces.

Iptables Configuration:

1. **Flush Existing Rules:**
 - Clear all current iptables rules to start with a clean configuration:

```
iptables -F
```
2. **Verify Rule List:**
 - Display the current iptables rules to confirm that the list is empty:

```
iptables -L
```

3. Add Necessary Rules:

a. Allow Loopback Interface Traffic:

- Permit all incoming traffic on the loopback interface to ensure internal system processes can communicate:

```
iptables -A INPUT -i lo -j ACCEPT
```

b. Allow Traffic from the Server's IP Address:

- Replace your_server_ip with the server's actual IP address:

```
iptables -A INPUT -s your_server_ip -j ACCEPT
```

c. Block Incoming Traffic on TCP Port 49999:

- Reject all incoming traffic on TCP port 49999:
- ```
iptables -A INPUT -p tcp --dport 49999 -j REJECT
```

*Note: This rule ensures that any external attempts to connect to port 49999 are denied, while internal traffic from the server is permitted.*

### 4. Confirm the Rules:

- List the current iptables rules to verify that the three rules have been added correctly:

```
iptables -L -n -v
```

### Test Plan:

- Utilize tools like ss to monitor and verify the connection states during testing.

By implementing these steps, you ensure that all traffic originating from the server is permitted across all ports and interfaces, while selectively blocking specific external traffic as defined.

| Test purpose                                                            | Test command | Expected test result | Actual test result |
|-------------------------------------------------------------------------|--------------|----------------------|--------------------|
| Client connection <u>from your server</u> using <i>loopback address</i> |              | OK                   |                    |
| Client connection <u>from your server</u> using <i>static IP</i>        |              | OK                   |                    |
| Client connection <u>from your client</u> using <i>static IP</i>        |              | NOK                  |                    |

Note: Table provided for your convenience only.

#### **Scenario #4: Permit Incoming Traffic for a Specific Service to All Hosts Except One in a Subnet**

In this scenario, we aim to configure the server to accept incoming traffic on port 49999 from all hosts within the 172.16.31.0/24 subnet, except for a specific client host.

##### **Server Setup:**

###### **1. Initialize the Server:**

- Configure the server to listen on port 49999 across all network interfaces:

```
nc -vkl 49999
```

*Note: The -v option enables verbose output, -k keeps the server open for multiple connections, and -l sets it to listening mode.*

###### **2. Verify Listening Status:**

- Ensure the server is actively listening on the specified port:

```
ss -tuln | grep 49999
```

*This command checks for TCP sockets in listening state on port 49999.*

##### **Iptables Configuration:**

###### **1. Review Current Rules:**

- Display the existing iptables rules to understand the current configuration:

```
iptables -L -n -v
```

*You should observe three rules: two ACCEPT rules and one REJECT rule.*

###### **2. Insert Specific Rules:**

###### **a. Block Incoming Traffic from the Specific Client on Port 49999:**

- Determine the IP address of the client to be blocked, referred to here as client\_ip.
- Insert a rule to reject incoming traffic from this client on port 49999 as the third rule:

```
iptables -I INPUT 3 -s client_ip -p tcp --dport 49999 -j REJECT
```

The `-I INPUT 3` option inserts this rule at the third position in the `INPUT` chain.

**b. Allow Incoming Traffic from the Subnet on Port 49999:**

- Insert a rule to accept incoming traffic from the 172.16.31.0/24 subnet on port 49999 as the fourth rule:

```
iptables -I INPUT 4 -s 172.16.31.0/24 -p tcp --dport 49999 -j ACCEPT
```

This rule is inserted at the fourth position to ensure proper rule evaluation order.

*Note: The insertion order is crucial. Inserting these rules ensures they are evaluated before the existing REJECT rule, allowing the specified subnet while blocking the specific client.*

**3. Verify the Updated Rules:**

- List the iptables rules to confirm the correct order and configuration:

```
iptables -L -n -v --line-numbers
```

You should see five rules, with the newly inserted rules in the correct positions.

**Test Plan:**

- Utilize tools such as `ss` to monitor connection states during testing.
- Attempt to connect to port 49999 from various hosts within the 172.16.31.0/24 subnet, including the blocked client, to verify that the iptables rules function as intended.

By following these steps, the server will accept incoming traffic on port 49999 from all hosts within the specified subnet, except for the designated client, effectively implementing selective access control.

| Test purpose                                                            | Test command | Expected test result | Actual test result |
|-------------------------------------------------------------------------|--------------|----------------------|--------------------|
| Client connection <u>from your server</u> using <i>loopback address</i> |              | OK                   |                    |
| Client connection <u>from your server</u> using <i>static IP</i>        |              | OK                   |                    |
| Client connection <u>from your client</u> using <i>static IP</i>        |              | NOK                  |                    |

Note: Table provided for your convenience only.

## Summary of Configured iptables Rules:

The current iptables rules are structured in the following sequence to manage incoming traffic effectively:

### 1. Allow Traffic Originating from the Server:

- Two rules are implemented to permit all incoming and outgoing traffic on the server's loopback interface (lo), ensuring internal communications function correctly.

### 2. Block Traffic from a Specific Client:

- A rule is set to reject incoming traffic from a designated client IP address on port 49999, effectively preventing this client from accessing the specified service.

### 3. Allow Traffic from Other Hosts in the Client Subnet:

- A rule allows incoming traffic on port 49999 from all other hosts within the 172.16.31.0/24 subnet, ensuring these hosts can access the service.

### 4. Block Traffic from All Other Hosts:

- A default rule is established to reject incoming traffic on port 49999 from any hosts not previously specified, enhancing the server's security by limiting access.

This rule order ensures that traffic from the server and trusted hosts is permitted, while undesired connections are effectively blocked.

#### Iptables listing

```
ACCEPT all anywhere anywhere
AECCEPT all server_host anywhere
REJECT tcp client_host anywhere tcp dpt:49999 ...
ACCEPT tcp 172.16.31.0/24 anywhere tcp dpt:49999 ...
REJECT tcp anywhere anywhere tcp dpt:49999 ...
```

## Managing firewall rules

In Red Hat Enterprise Linux 8, while firewalld with nftables is the default firewall management tool, iptables remains available for compatibility. If you choose to manage firewall rules using iptables, it's crucial to ensure these rules persist across system reboots. Here's how you can manage, save, and restore iptables rules:

### Saving and Restoring iptables Rules

1. **Saving Rules:** iptables rules are stored in memory and will be lost after a reboot. To save the current rules to a file, use the iptables-save utility:

```
iptables-save > /etc/sysconfig/iptables
```

This command saves the current iptables rules to the /etc/sysconfig/iptables file, ensuring they can be reloaded upon system startup.

2. **Restoring Rules:** To restore iptables rules from a saved file, use the iptables-restore utility:

```
iptables-restore < /etc/sysconfig/iptables
```

This command reloads the rules from the specified file into the current iptables configuration.

## Ensuring iptables Rules Persist After Reboot

To maintain iptables rules after a system reboot, follow these steps:

1. **Install iptables-services:** The iptables-services package allows systemd to manage iptables rules:

```
dnf install iptables-services
```

This package provides the necessary scripts to save and restore iptables rules during system shutdown and startup.

2. **Disable firewalld and Enable iptables:** Since firewalld is the default firewall management tool in RHEL 8, you'll need to disable it and enable the iptables service:

```
sudo systemctl stop firewalld
sudo systemctl disable firewalld
sudo systemctl enable iptables
sudo systemctl start iptables
```

These commands stop and disable firewalld, then enable and start the iptables service, ensuring that your iptables rules are applied at boot.

## Creating a Firewall Script for iptables Rules

For better management and version control, consider storing your iptables rules in a shell script. This approach allows for easier modifications and documentation. Here's how you can create such a script:

1. **Create the Script File:** Use your preferred text editor to create a new shell script file, e.g., iptables\_rules.sh.
2. **Add Shebang and Header Information:** At the top of the script, add the shebang line and any necessary comments.

3. **Define Variables (Optional):** Set variables for commonly used IP addresses, interfaces, or ports to simplify rule definitions.
4. **Flush Existing Rules:** Ensure that previous rules do not interfere with the new ones.
5. **Add Your iptables Rules:** Include all the necessary iptables rules.
6. **Make the Script Executable:** Set the appropriate permissions to execute the script.
7. **Execute the Script:** Run the script to apply the iptables rules.

## Managing the iptables Service

The iptables service, when started, loads the system firewall rules stored in `/etc/sysconfig/iptables`. To manage this service:

- **Start the Service:**

```
systemctl start iptables
```

- **Enable the Service to Start on Boot:**

```
systemctl enable iptables
```

## Section D - Utilizing Network Utilities

### Working with tcpdump

#### Overview

The tcpdump utility is a powerful command-line packet analyzer used to capture and inspect network traffic in real-time.

#### Syntax

The basic syntax of tcpdump is:

```
tcpdump [options] [expression]
```

#### Commonly Used Options:

- **-D:** List available interfaces.
- **-i interface:** Specify the interface to listen on.
- **-n:** Display numerical addresses instead of resolving hostnames.
- **-q:** Provide less verbose output.

- -X: Display packet contents in both hex and ASCII.
- -tttt: Include a readable timestamp.
- {dst|src} {host IP|port number|portrange range}: Define the source or destination for filtering.
- -w file: Write captured packets to a file.
- -r file: Read packets from a file.

## Examples

1. **Capture packets on a specific interface from/to a server IP address:**

```
tcpdump -i ens224 host 172.16.30.167
```

2. **Capture packets with numeric display of hosts and services:**

```
tcpdump -i ens224 -qn host 172.16.30.167
```

3. **Capture packets with a well-formatted timestamp:**

```
tcpdump -i ens224 -qn -tttt host 172.16.30.167
```

4. **Capture packets and display contents in hex and ASCII:**

```
tcpdump -i ens244 -qnX -tttt host 172.16.30.167
```

5. **Capture packets from/to a specific port:**

```
tcpdump -i ens224 -qnX -tttt host 172.16.30.167 and port 22
```

6. **Capture packets using a specific network protocol:**

```
tcpdump -i ens224 -qnX -tttt host 172.16.30.167 and arp
```

## Exercise: Capture Traffic on Server & View Data Sent from Client

1. **On the server:** Execute tcpdump with the following options to capture traffic on the "ens224" interface from/to your server, display numeric values for hosts and services (port numbers), display ASCII output, and include a readable timestamp:

```
tcpdump -i ens224 -qnX -tttt host <server_ip>
```

2. **On the server:** Start a Netcat (nc) server on port 59999:

```
nc -l 59999
```

3. **On the client:** Connect to the Netcat server using an nc client:

```
nc <server_ip> 59999
```

4. **Interaction:** Exchange messages between the server and client. Observe the clear-text messages displayed in your tcpdump output.